Where AI Agents Break:
An Independent Study of AI Agents on Realistic SaaS Revenue Operations Tasks

Author: Wasif Saeed

Research artifacts: samples/, logs/, report/

Model evaluated: gemini/gemini-3.5-flash via terminus-2

Difficulty-calibration goal: Below 30% aggregate task-level pass@3

Research domain: SaaS finance and Revenue Operations data-science workflows

Executive Summary

This study presents a five-task Harbor-format benchmark focused on realistic SaaS Revenue Operations work: MRR reconstruction, pricing-migration audits, sales-commission reconciliation, temporal-leakage audits, and usage-billing dispute investigations.

The slice is intentionally narrow enough to be coherent, but broad enough to exercise practical data-science skills that matter in production: structured ETL, policy-driven rule application, spreadsheet formula interpretation, PDF extraction, multi-source reconciliation, in-place workbook repair, and multi-module source-code audit.

All five tasks pass the sanity gates: the oracle agent achieves reward 1.0, and the nop agent achieves reward 0.0. Across 15 Gemini trials, two trials passed. The aggregate trial-level pass@1 is 13.3%, and the aggregate task-level pass@3 is 20.0%. This places the benchmark within its intended difficulty range while preserving one intentionally easier baseline task.

The difficulty curve is useful because it is not flat. structured-mrr-rebuild is the structured-only baseline and passes 2 of 3 Gemini trials. The other four tasks pass 0 of 3 Gemini trials because they add harder representation or source-audit layers: Excel formula input cells, multi-sheet workbook repair, formula-controlled artifact preservation, multi-module ML pipeline leakage inspection, PDF invoice extraction, and FX-aware reconciliation.

The dominant failures are realistic agent mistakes rather than broken environments: reading Excel formulas as NaN, repairing only part of a workbook, leaving leaky helper functions in source, applying FX to the wrong side of a reconciliation, or missing one-cent rounding semantics in derivative metrics.

1. Distribution: Why This Slice?

1.1 Chosen slice

The selected slice is SaaS Revenue Operations and operational analytics data science. I chose this slice because it represents a common but under-benchmarked part of real industry data-science work: turning messy operational data into reliable business-critical outputs.

Instead of focusing on novel model training, this slice focuses on the kinds of tasks data scientists and analytics engineers often handle inside finance, billing, sales operations, and customer analytics teams. These workflows require agents to interpret business rules, reconcile multiple sources of truth, work with imperfect files, and produce outputs that downstream teams can trust.

The final task distribution covers five related SaaS workflows:

- Rebuilding monthly recurring revenue from raw billing exports.
- auditing a pricing-migration rule engine;
- reconciling sales commission statements;
- auditing a revenue-retention model for temporal leakage;
- investigating customer usage-billing disputes.

This distribution was chosen to stay inside one coherent industry domain while still testing different capabilities. The MRR task acts as the structured ETL baseline. The pricing and quota tasks test rule application, Excel formula reasoning, and workbook repair. The billing dispute task adds PDF invoice extraction and multi-currency reconciliation. The retention task adds an ML pipeline audit where the core challenge is temporal feature availability rather than model architecture.

1.2 Where the task distribution comes from

The distribution comes from production-style SaaS analytics and RevOps workflows rather than toy data-science exercises. These are the kinds of problems that appear in internal finance dashboards, billing audits, sales compensation reviews, pricing migrations, customer churn analysis, and post-hoc model audits.

I also drew on established benchmark-design principles that prioritize realistic, messy inputs and executable outputs. Recent data-science and terminal-agent benchmarks emphasize production-style ETL, multi-modal files, partially corrupted data, structured schemas, and artifact editing.

For this reason, the study includes CSV, JSONL, SQLite, PDF, and Excel inputs. However, the benchmark is organized around real operational workflows rather than file formats: revenue

reconstruction, billing correctness, commission reconciliation, pricing-migration validation, and leakage-safe retention modelling.

1.3 What this slice measures

This slice measures whether an agent can perform operational data-science work where correctness depends on business semantics, not just code generation.

The tasks are designed to measure:

- rule interpretation from business memos, policy PDFs, and contract-like documents;
- multi-source reconciliation across billing, usage, customer, quota, invoice, and workbook data;
- spreadsheet reasoning, including formula-backed cells, stale ranges, hidden sheets, and workbook preservation;
- exact schema compliance for CSV, JSON, parquet, and XLSX outputs;
- source-code repair in existing data pipelines and rule engines;
- temporal reasoning in ML feature pipelines;
- deterministic verification through Harbor tests rather than subjective grading.

1.4 What this slice does not measure

The slice does not attempt to cover all of data science. It intentionally does not measure broad DS areas such as computer vision, NLP model training, recommender systems, experimental design, A/B testing, deep learning optimization, scientific computing, geospatial analysis, or open-ended dashboard exploration.

It also does not focus on Kaggle-style modelling performance, where the main goal is maximizing a metric through feature engineering and model selection. The one ML task in the portfolio is included because temporal leakage is a real production DS failure mode, not because the eval is about training the strongest churn model.

This boundary keeps the eval focused. The goal is to test a narrow, realistic industry slice where current agents can appear competent but still fail on details that matter in production: exact business rules, source-of-truth conflicts, spreadsheet semantics, temporal availability, and financial reconciliation.

2. Task Portfolio

Task	Real-world workflow	Main input types	Required outputs	Capability isolated
structured-mrr-rebuild	Rebuild SaaS MRR from raw billing exports	CSV, JSONL	Parquet fact table, JSON summary, monthly CSV	Structured ETL baseline and business-rule inference
pricing-migration-rule-engine-audit	Repair a SaaS pricing-migration audit engine	CSV, JSONL, PDF, XLSX, Python code	3 CSVs + 1 JSON	Multi-module code repair and exception workbook evaluation
quota-commission-reconciliation-audit	Repair a Q1 sales-commission audit workbook	CSV, JSONL, PDF, XLSX	Repaired XLSX + JSON	In-place workbook repair and formula-semantics reasoning
revenue-retention-leakage-audit	Audit a churn model feature pipeline for temporal leakage	SQLite, PDF, CSV, Python code	JSON audit, CSV backtest, parquet predictions, feature list	Temporal leakage and source-code excision
usage-billing-dispute-audit	Investigate customer usage billing disputes	PDFs, CSV, XLSX	Five-sheet XLSX audit workbook	Multi-modal invoice parsing and FX-aware reconciliation

2.1 Structured MRR rebuild

This is the structured ETL baseline. The agent rebuilds MRR by account and month from customers.csv and subscriptions.jsonl, invoice_lines.csv, and fx_rates.csv.

The task requires service-month revenue recognition, annual and quarterly normalization, line-type filtering, tax exclusion, FX conversion, deduplication, negative credits, and summary metrics. It has 25 accounts, 60 raw invoice rows, 46 expected account-month outputs, and 27 deterministic tests.

It is intentionally easier than the others: no PDF parsing, no Excel formulas, no codebase repair, and no in-place artifact editing. Gemini passed 2 of 3 trials; the single failure was a one-cent month-over-month rounding boundary.

2.2 Pricing migration rule-engine audit

This task asks the agent to repair a broken Python rule engine for a SaaS plan migration. Inputs include account CSVs, product usage events, contract JSONL, a policy PDF, new plan assignments, and a multi-sheet exception_approvals.xlsx workbook.

The hard residual trap is the exception approval workbook: real headers are offset, decoy sheets contain stale literals, and valid approvals include formula-backed approved plans and prices. Gemini repaired much of the rule engine but failed all three trials on formula-backed approval values and downstream aggregates.

2.3 Quota commission reconciliation audit

This task simulates a sales-commission reconciliation for 20 account executives. The agent must compute correct commissions from a PDF plan, quota CSV, multi-sheet carryover workbook, JSONL bookings, and draft commission statements.

The output is an existing Excel audit workbook repaired in place. The workbook begins as a stale 8-rep template and must be expanded to 20 reps while preserving formulas, hidden sheets, named ranges, conditional formatting, data validations, helper rows, table ranges, and reviewer notes.

Gemini passed 0 of 3 trials. Common failures include leaving the worksheet-level auto_filter.ref stale, using Python-style rounding instead of Excel rounding, and applying a uniform ROUND formula where specific reps use MAX floors.

2.4 Revenue retention leakage audit

This is the portfolio's ML task. The agent receives a SQLite warehouse, a feature-availability policy PDF, backtest windows, a neutral readme, and a six-module Python feature pipeline. The task is to remove or repair temporally leaky features, run honest rolling backtests, and produce April 2026 holdout predictions.

The pipeline uses obfuscated feature names f01 through f13, so the agent must map code behaviour to feature families instead of relying on obvious names. The verifier checks final outputs and source-code hygiene: leaky helper functions and horizon-inclusive patterns must disappear from /root/project.

Gemini passed nearly all functional tests but failed source-integrity and audit-documentation gates in all three trials.

2.5 Usage billing dispute audit

This task models a customer billing dispute investigation. It includes three invoice PDFs with different layouts, a pricing addendum PDF, usage logs, customer registry, FX rates, support tickets, and a legacy Excel adjustment workbook. The output is a five-sheet Excel audit workbook.

The primary representation trap is an Excel input workbook whose formula cells have no cached values. Naive pandas or openpyxl data_only=True readers return NaN. The agent must inspect formula text and evaluate SUM / MAX style adjustments.

The repeated failure is multi-currency semantics: Gemini often parses invoices and formulas correctly, but misinterprets whether expected overage, billed overage, and discount deltas are USD- or EUR-denominated.

3. Difficulty Profile

3.1 Per-task results

Task	Oracle	Nop	Gemini trial rewards	pass@1	pass@3	Main outcome
structured-mrr-rebuild	1.0	0.0	1 / 0 / 1	0.667	1.000	Structured baseline solved in 2 of 3 trials
pricing-migration-rule-engine-audit	1.0	0.0	0 / 0 / 0	0.000	0.000	Formula-backed approval workbook defeated all trials
quota-commission-reconciliation-audit	1.0	0.0	0 / 0 / 0	0.000	0.000	Workbook repair and Excel formula semantics defeated all trials
revenue-retention-leakage-audit	1.0	0.0	0 / 0 / 0	0.000	0.000	Source-integrity and audit-documentation gates defeated all trials
usage-billing-dispute-audit	1.0	0.0	0 / 0 / 0	0.000	0.000	FX and discount reconciliation semantics defeated all trials

3.2 Aggregate results

- Total tasks: 5.
- Total Gemini trials: 15.
- Passing Gemini trials: 2.
- Aggregate trial-level pass@1: 2 / 15 = 13.3%.
- Aggregate task-level pass@3: 1 / 5 = 20.0%.
- Observed result: 20.0% task-level pass@3, within the intended sub-30% difficulty-calibration range.

3.3 Difficulty curve

The difficulty curve is intentionally uneven. `structured-mrr-rebuild` anchors the easy end because it is a structured ETL task built mostly around CSV and JSONL inputs. It still requires careful business-rule application, but it avoids harder representation layers such as PDF parsing, workbook repair, and source-code auditing.

The other four tasks increase difficulty by adding production-style complexity: formula-backed Excel exceptions, in-place workbook repair, multi-module source-code inspection, temporal leakage removal, PDF invoice extraction, FX-aware reconciliation, and dispute classification.

The curve should therefore be read as a qualitative difficulty progression. Gemini can sometimes solve the structured baseline, but failures dominate once the tasks require identifying authoritative sources, evaluating spreadsheet formulas, preserving artifacts, cleaning leaky source paths, or reconciling multi-currency billing logic.

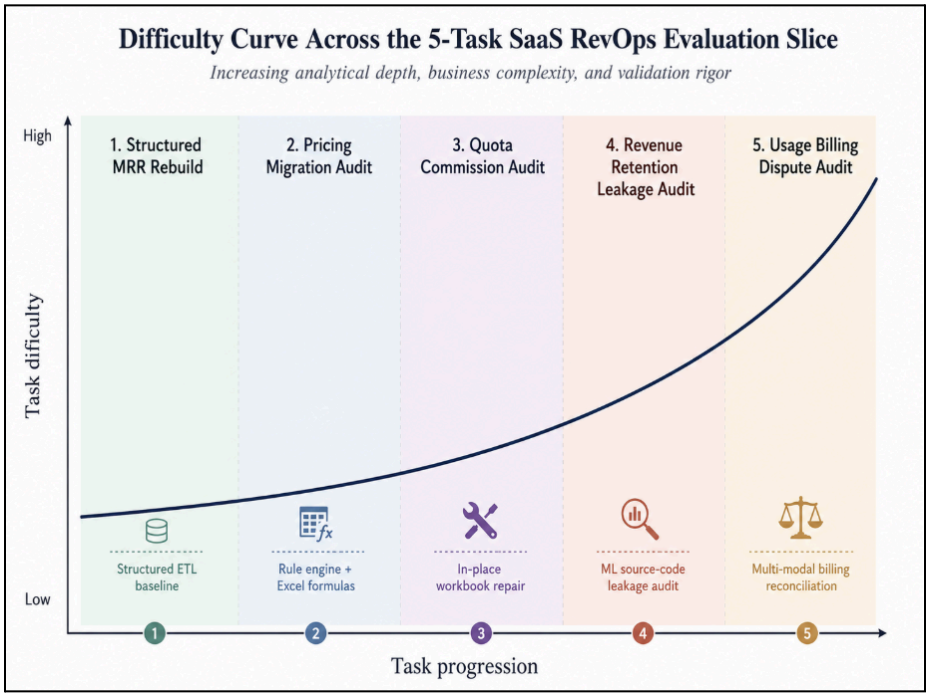

Figure 1: Qualitative difficulty progression across the five-task SaaS RevOps evaluation slice. The curve reflects added reasoning layers and observed failure modes rather than a continuous numeric difficulty score.

3.4 Dominant failure families

Failure family	Tasks affected	Representative example
Excel formula input cells with no cached value	pricing, quota, billing	Formula cells read as NaN unless inspected with formula text.
Workbook artifact repair	quota	Agent repairs table range but misses worksheet-level autofilter or hidden helper rows.
Multi-module source-code audit	retention leakage	Agent removes leaky outputs but leaves ts_he or pm_next helpers in source.
Cross-currency reconciliation semantics	billing	Agent stores EUR discount gap instead of full USD reconciliation delta.
Rounding or formula semantics	mrr, quota	One-cent MoM rounding; Excel ROUND vs Python round; MAX floor traps.

4. Failure Analysis From Trajectories

4.1 Structured MRR rebuild: baseline task with one precision miss

The failing MRR trial passed 26 of 27 tests. It inferred the main business rules: service-month FX, annual and quarterly normalization, negative credits, non-recurring exclusions, deduplication, catch-up billing, and $319.00 parsing.

The only failure was a one-cent March mrr_change_usd value: expected -2409.53, received -2409.52. This indicates the model had the ETL logic but made a derivative rounding-order or floating-point precision error. That is a genuine, small difficulty signal and supports interpreting this task as the baseline rather than the main hard task.

4.2 Pricing migration: lookup tables instead of authoritative formulas

Across all three pricing trials, Gemini fixed many code bugs: usage-window logic, grandfathering, Scale seat-floor pricing, violation taxonomy, exception review output, and revenue-impact signs. The repeated failure cluster came from exception_approvals.xlsx.

The model discovered the workbook and header row but used decoy or intermediate values instead of evaluating the authoritative formulas. For example, it treated Ref_X7.tgt_plan or archive literals as final, producing Launch where the formula-backed approval returns Scale. It also read stale literal prices instead of evaluating formula cells that produce 3500.00.

This is fair: the formulas and sheets are visible in the workbook, openpyxl is installed, and the oracle proves the formulas can be evaluated in the environment.

4.3 Quota commission: workbook repair and formula semantics mistakes

Quota failures are split into two clusters. First, all three trials missed the worksheet-level auto_filter.Ref: agents repaired the Excel table reference but not the separate worksheet filter. Second, agents often miscompute carryover formulas. Some applied uniform ROUND formulas and missed MAX floors for REP-012 and REP-015; another used Python rounding where Excel ROUND semantics were needed.

The agents generally understood the commission plan and repaired much of the workbook, but missed subtle Excel artifacts. This is a strong, hard-task signal because real finance templates often contain separate formula cells, tables, filters, hidden helper ranges, and named ranges.

4.4 Revenue retention leakage: output-correct but source-not-clean

In two trials, Gemini produced nearly correct outputs and passed 54 of 55 tests, but failed source-integrity checks because label-horizon helpers remained in period_utils.py. In another trial, it documented honest alternate features f12 and f13 incorrectly in leakage_audit.json.

This failure is valuable because it distinguishes clean final matrices from full source sanitization. In production ML audits, dormant leaky helpers matter because they can be reactivated later; the verifier intentionally checks patched source as well as final outputs.

4.5 Usage billing dispute: correct extraction, wrong FX semantics

The billing trials produced valid workbooks and frequently evaluated the legacy Excel formula input cells correctly. The repeated failure was the USD/EUR reconciliation semantics for acct_002.

One trial incorrectly converted expected USD overage by FX. Two others computed correct total deltas but wrote the pre-FX EUR discount gap into discount_delta_usd instead of the full USD reconciliation delta. This is a realistic RevOps failure: agents can parse PDFs and formulas but still misinterpret which side of a reconciliation is denominated in which currency.

4.6 Why these are genuine failures, not task bugs

The failure evidence supports the task design. Every task has an oracle reward of 1.0 and a nop reward of 0.0. Failures are narrow and trajectory-explainable, not environment crashes. Agent outputs often pass most tests before failing on realistic edge cases. The failing constants trace to agent-visible data, policy PDFs, workbook formulas, or source code. No task relies on LLM-as-judge behaviour.

5. Research Awareness

This design was informed by recent data-science agent benchmarks, executable terminal-task evaluation, production analytics practices, and repair-oriented software benchmarks. The main takeaway from the research was that realistic data-science evaluation should test messy, end-to-end work rather than isolated textbook analysis.

DSAEval and DSBench emphasize that real data-science tasks involve long-context reasoning, multiple files, mixed data formats, iterative exploration, and executable outputs rather than single-step answers [1, 2]. That influenced the decision to build tasks around CSV, JSONL, PDF, XLSX, SQLite, parquet, and repaired source files. DA-Code also supports evaluating data-science agents through code execution and structured artifacts, which is why the tasks require files that can be checked deterministically instead of prose summaries [4].

MLE-bench informed the inclusion of one ML-oriented task, but I intentionally did not make the full slice about model training [3]. The revenue-retention task focuses on temporal leakage and feature availability, because that is a common production ML failure mode. This design is also supported by scikit-learn's TimeSeriesSplit guidance, which motivates chronological validation for time-dependent data instead of random splits [6]. Feast's point-in-time feature retrieval documentation also reinforces the importance of respecting feature availability when building ML datasets [13].

Terminal-Bench and Harbor-style task design influenced the evaluation harness: each task has a sandboxed environment, executable agent actions, and a final-state verifier [5]. This pushed the design toward deterministic pytest checks over LLM-as-judge scoring. In this slice, correctness depends on exact files, schemas, workbook ranges, source-code patterns, and dollar-level outputs.

Repair-oriented benchmarks also influenced the task design. SWE-bench evaluates agents on real GitHub issues where the model must modify an existing codebase rather than write a new script from scratch [12]. Defects4J follows a similar principle by collecting reproducible real software faults for controlled repair studies [17]. These benchmarks support the idea that repair tasks are often more realistic than clean-slate generation because real engineering and analytics work usually happens inside existing systems. This influenced the inclusion of tasks where agents must repair a rule engine, preserve an existing Excel workbook, and remove leakage from an existing ML pipeline.

The spreadsheet-heavy tasks were shaped by openpyxl and pandas behaviour. openpyxl's data_only mode depends on stored cached workbook values, and generated formula cells may not have cached results [7]. pandas read_excel can therefore surface formula-backed cells as missing

values in these cases [8]. This behaviour directly motivated the Excel formula traps in the pricing, quota, and billing tasks.

The production workflow mapping came from SaaS analytics and RevOps systems. MRR rebuilds resemble ChartMogul, Baremetrics, and dbt-style revenue analytics workflows [14]. Usage-billing disputes resemble Stripe Billing, Chargebee, and Zuora-style metered billing reconciliation workflows [15, 16]. Commission audits resemble sales-compensation workflows found in tools such as CaptivateIQ, Xactly, Spiff, and Anaplan. The task designs do not depend on these vendor APIs, but the workflows informed the distribution: revenue reconstruction, pricing migration, commission reconciliation, usage dispute investigation, and point-in-time retention modeling.

The broader data-engineering references also shaped the slice. The Data Cube paper connects to aggregation-heavy analytics tasks such as MRR summaries, commission totals, and billing reconciliation rollups [9]. Designing Data-Intensive Applications and The Data Warehouse Toolkit informed the emphasis on reliable data pipelines, source-of-truth consistency, schema discipline, and dimensional business reporting [10, 11].

Overall, the research takeaway was clear: useful headroom is not only found in asking agents to solve toy data questions or maximize a model score. It is found in testing whether agents can handle realistic operational analytics where correctness depends on business semantics, messy representations, existing artifacts, source-of-truth conflicts, spreadsheet behaviour, temporal boundaries, and deterministic verification.

6. Scale Plan: From 5 Tasks to 1,000

The scaling plan should preserve the same principle as the final five tasks: generator-built, oracle-solvable, nop failing tasks with trajectory-explainable failures. Scaling should not mean hand-writing 1,000 one-off prompts; it should mean expanding a controlled generator library.

6.1 Generator-first architecture

- Parameterized input generators for CSV, JSONL, PDF, XLSX, SQLite, and source-code stubs.
- A deterministic oracle generated from the same seed.
- A verifier generated from seed-level expected values, without exposing answer constants to the agent.
- Anti-naive probes that intentionally apply common wrong strategies.
- Metadata that records active traps, input modalities, expected difficulty band, and dominant failure family.

6.2 Task-family expansion

Family	Current seed task	Scaling plan
MRR / billing ETL	structured-mrr-rebuild	Vary accounts, currencies, billing intervals, FX volatility, credits, duplicate density, and revenue-recognition boundaries.
Pricing migration	pricing-migration-rule-engine-audit	Vary plan maps, exception types, formula workbook structures, usage thresholds, and policy precedence rules.
Commission reconciliation	quota-commission-reconciliation-audit	Vary rep counts, tier thresholds, commission rates, stale workbook structures, and formula patterns.
ML leakage audit	revenue-retention-leakage-audit	Vary feature families, obfuscated columns, temporal grains, source modules, and leakage patterns.
Usage billing dispute	usage-billing-dispute-audit	Vary invoice layouts, account counts, currencies, pricing tiers, taxes, support-ticket mappings, and adjustment formulas.

6.3 QA and curation loop

1. Generate a candidate task from a seed and parameter set.
2. Run the oracle gate; the oracle must reach reward 1.0.
3. Run the nop gate; the nop agent must receive a reward of 0.0.
4. Run scripted naive solutions to confirm the verifier catches common shortcuts.
5. Run at least three Gemini trials on a sample from each difficulty band.
6. Review trajectories and reject tasks where failures come from ambiguity, missing dependencies, or verifier-only assumptions.
7. Balance the released set so that no single trap dominates the full benchmark.

6.4 Expected scaling risks and mitigations

Risk	Mitigation
Overfitting to one representation trap, especially Excel formulas.	Maintain a target mix across structured ETL, PDF parsing, workbook repair, source audit, and temporal validation.
Hidden verifier-only assumptions.	Require every expected value to trace to instruction, data, policy PDF, visible workbook, or source code.
Skills becoming answer keys.	Keep skills methodology-only and audit them for leaked constants or trap-specific recipes.
Documentation drifting from implementation.	Generate task cards directly from seed metadata and validate them in CI.
Binary reward hiding useful near-miss information.	Keep binary final reward, but store per-test results, trajectory tags, and failure-family labels for training analysis.

7. Verifier Design and Quality Controls

The verifiers are deterministic pytest-style checkers. They inspect the final container state: output files, schemas, numeric values, workbook artifacts, and patched source code. No task uses LLM-as-judge scoring.

7.1 Why binary reward?

Binary reward is appropriate for this study because these tasks represent operational workflows where one missed artifact can break downstream finance or audit usage. Partial progress is still visible through pytest output and trajectory analysis, but the Harbor reward remains clear and reproducible.

7.2 Anti-naive tests

- Check known wrong FX joins, including invoice-month FX instead of service-month FX.
- Reject naive Excel reads that return NaN for formula cells with no cached values.
- Check workbook artifacts separately, including worksheet autofilter, table ranges, named ranges, validations, conditional formatting, and hidden helper rows.
- Inspect patched source code for leftover leaky helper functions or horizon-inclusive logic.
- Test sign conventions and currency semantics directly in reconciliation outputs.

7.3 Fairness posture

The tasks were designed so that failures can be attributed to genuine difficulty. Each task has an oracle pass and nop fail. Expected values are derivable from visible inputs. Failure clusters are narrow and repeatable. The verifiers are strict but not arbitrary: they test properties that would matter in production, such as exact schema contracts, spreadsheet preservation, and point-in-time feature availability.

8. Limitations

- The portfolio is deliberately narrow. It covers SaaS finance and RevOps workflows, not all of data science.
- Four of the five tasks have 0% pass@3, so a future version should add medium-difficulty tasks between the baseline and the hardest representation traps.
- The tasks use synthetic data, even though the patterns are realistic. Synthetic generation is useful for deterministic verification, but public-data variants would improve external validity.
- Binary reward is easy to audit, but near-miss trajectories should still be used for model-training signal and task-quality review.
- The benchmark currently evaluates gemini-3.5-flash via terminus-2 only. Other agents may expose different failure modes.

9. Conclusion

This five-task pilot study is coherent, reproducible, and grounded in realistic operational workflows. It achieved an aggregate task-level pass@3 of 20.0%, placing it within the intended sub-30% difficulty-calibration range, while the oracle and nop sanity checks behaved correctly across every task.

The evaluation is strongest where it isolates practical, high-value data-science failures: formula-backed spreadsheets, in-place workbook repair, multi-module source-code leakage, PDF and CSV reconciliation, FX semantics, and exact output contracts. These are not toy coding puzzles. They are the kinds of messy operational analytics tasks that data scientists and analytics engineers encounter in production.

The next step is to scale the generator approach carefully: maintain deterministic oracles, run nop and naive probes, preserve traceability from verifier constants to visible inputs, and curate a balanced difficulty curve rather than maximizing hardness alone.

References

Benchmarks and research context

[1] Sun et al. DSAEval: Evaluating Data Science Agents on a Wide Range of Real-World Data Science Problems. arXiv, 2026. https://arxiv.org/abs/2601.13591

[2] Jing et al. DSBench: How Far Are Data Science Agents to Becoming Data Science Experts? arXiv, 2024. https://arxiv.org/abs/2409.07703

[3] Chan et al. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering. arXiv, 2024. https://arxiv.org/abs/2410.07095

[4] Huang et al. DA-Code: Agent Data Science Code Generation Benchmark for Large Language Models. arXiv, 2024. https://arxiv.org/abs/2410.07331

[5] Terminal-Bench authors. Terminal-Bench: Benchmarking Agents on Hard, Realistic Terminal Tasks. arXiv, 2026. https://arxiv.org/abs/2601.11868

[6] scikit-learn documentation. TimeSeriesSplit. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html

[7] openpyxl documentation and usage notes. Workbook formula cells and data_only behavior. https://openpyxl.readthedocs.io/

[8] pandas documentation. pandas.read_excel. https://pandas.pydata.org/docs/reference/api/pandas.read_excel.html

[9] Jim Gray et al. Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals. Data Mining and Knowledge Discovery, 1997.

[10] Martin Kleppmann. Designing Data-Intensive Applications. O'Reilly Media, 2017.

[11] Ralph Kimball and Margy Ross. The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling. Wiley, 2013.

[12] Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv, 2023. https://arxiv.org/abs/2310.06770

[13] Feast documentation. Point-in-time feature retrieval and feature stores. https://docs.feast.dev/

[14] dbt Labs documentation. Analytics engineering and data modeling documentation. https://docs.getdbt.com/

[15] Stripe documentation. Billing, invoices, and metered usage documentation. https://docs.stripe.com/billing

[16] Zuora documentation. Billing and revenue operations documentation. https://knowledgecenter.zuora.com/

[17] Just, R., Jalali, D., and Ernst, M. D. Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs. ISSTA, 2014. https://doi.org/10.1145/2610384.2628055